

Kiểm Thử Dòng Điều Khiển

Khoa CNTT- Trường Đại học Công nghệ - ĐGQGHN

Nội dung chính

- Giới thiệu
- Đồ thị dòng điều khiển
- Đường đi trong đồ thị
- Độ phủ
- Kiểm thử dựa trên độ phủ
 - Độ phức tạp Cyclomatic

Giới thiệu

- Tên gọi: Glass box, structural testing, clear box testing, open box testing, **white box testing**
- Tiếng Việt: Kiểm thử cấu trúc, **kiểm thử hộp trắng**
- Là kỹ thuật dựa trên **mã nguồn** để xây dựng các ca kiểm thử và kiểm tra đầu ra
- Giúp phát hiện các lỗi/khiếm khuyết tiềm ẩn bên trong chương trình
- **Không thể thay thế kiểm thử hộp đen** → Kết hợp với kiểm thử hộp đen tạo thành một quy trình kiểm thử thống nhất

Giới thiệu

- Dựa trên các thông tin về:
 - Phân tích luồng điều khiển, luồng dữ liệu
 - Phân tích đồ thị, đường đi
 - Tiêu chuẩn bao phủ
- **Kiểm thử dòng điều khiển:** tập trung kiểm thử tính đúng đắn của các giải thuật
- **Kiểm thử dòng dữ liệu:** tập trung kiểm thử tính đúng đắn của việc sử dụng các biến dữ liệu trong chương trình

Kiểm Thử Dòng Điều Khiển

Đồ thị dòng điều khiển (Control Flow Graph)

- Định nghĩa:

CFP là một đồ thị có hướng của chương trình, trong đó:

- + **Đỉnh**: Lệnh (statement)/ khối lệnh (basic block)

- + **Cạnh**: Luồng điều khiển (control flow paths)

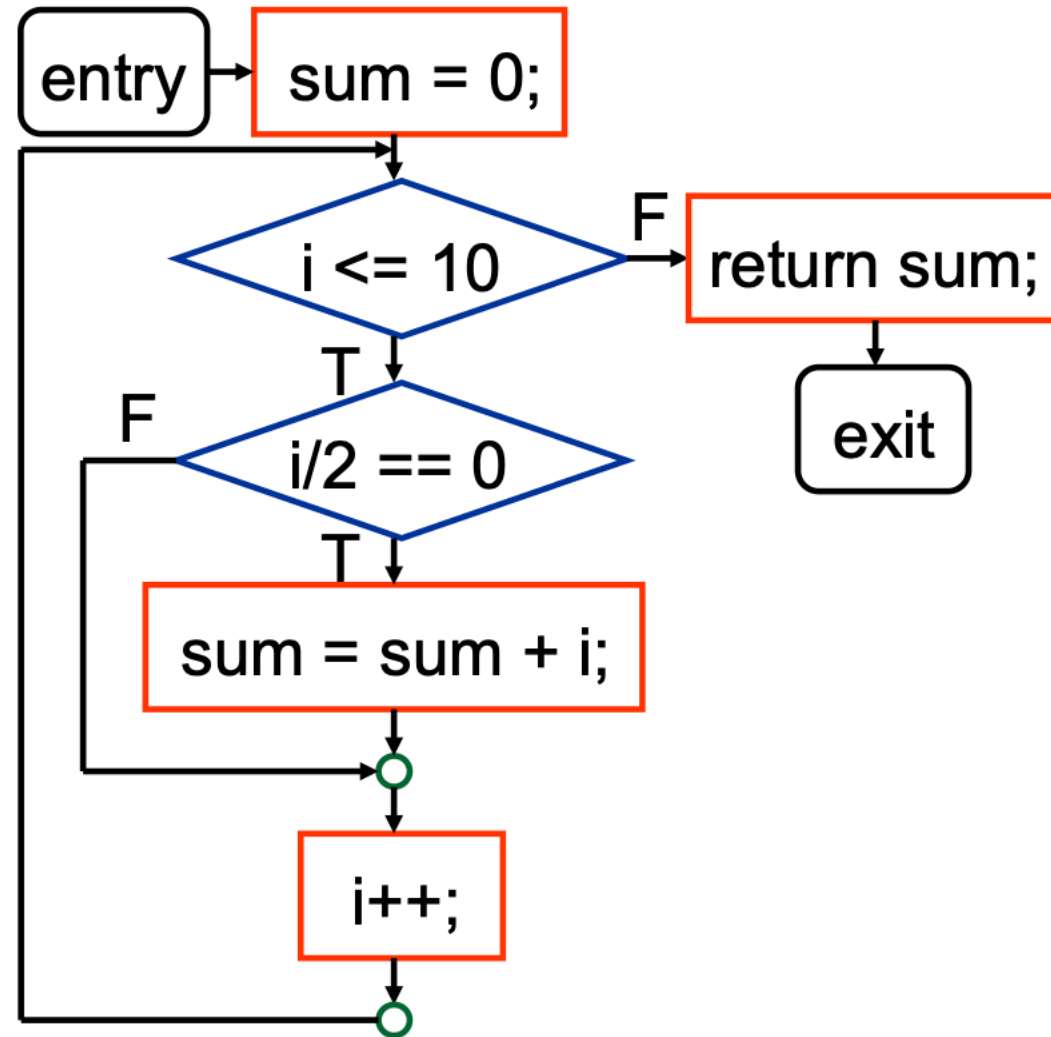
- + Nếu i và j là 2 đỉnh của đồ thị, tồn tại 1 cạnh từ i tới j nếu lệnh j có thể được thực hiện ngay sau lệnh i

- CFG biểu diễn tất các đường đi có thể trong chương trình (all possible execution paths)

CFG – Ví dụ

```
int evensum(int i)
{
    int sum = 0;

    while (i <= 10) {
        if (i/2 == 0)
            sum = sum + i;
        i++;
    }
    return sum;
}
```



CFG – Ví dụ khối cơ bản

FindMean (FILE ScoreFile)

```
{ float SumOfScores = 0.0;  
  int NumberOfScores = 0;  
  float Mean=0.0; float Score;  
  Read(ScoreFile, Score);
```

```
  2 while (! EOF(ScoreFile) {
```

```
    3 if (Score > 0.0 ) {
```

```
      SumOfScores = SumOfScores + Score;  
      NumberOfScores++;
```

```
    5 }
```

```
      Read(ScoreFile, Score);
```

```
    }
```

```
    /* Compute the mean and print the result */
```

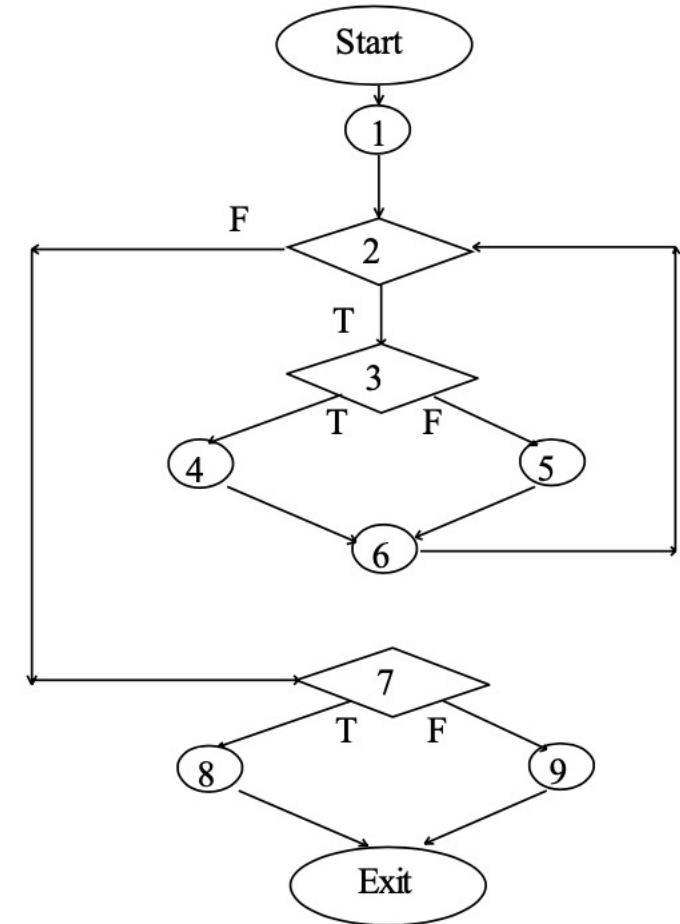
```
    7 if (NumberOfScores > 0) {
```

```
      Mean = SumOfScores / NumberOfScores;  
      printf(" The mean score is %f\n", Mean);
```

```
    } else
```

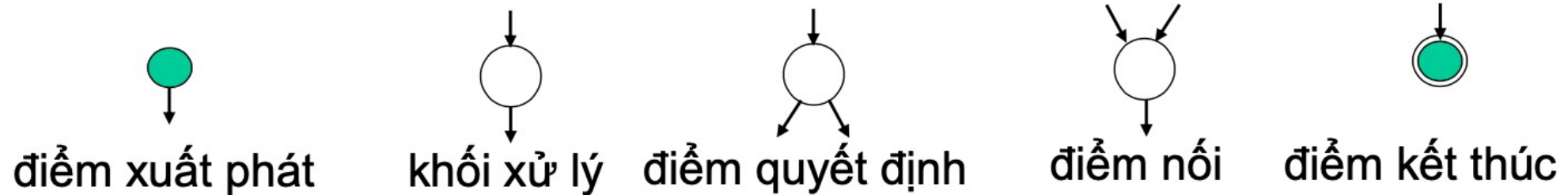
```
      printf ("No scores found in file\n");
```

```
  }
```

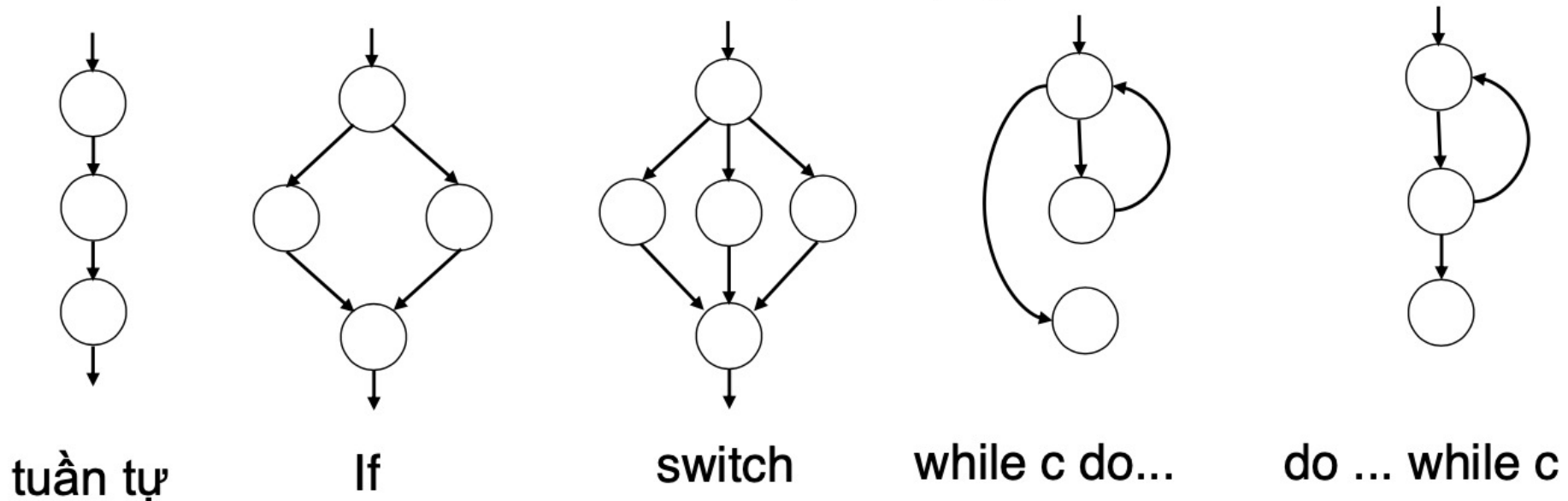


CFG – Các thành phần cơ bản

- Các thành phần cơ bản :

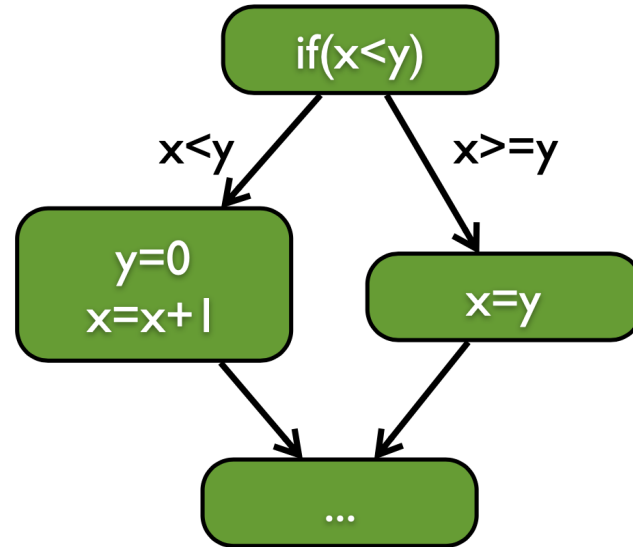


- Miêu tả các cấu trúc điều khiển phổ dụng :

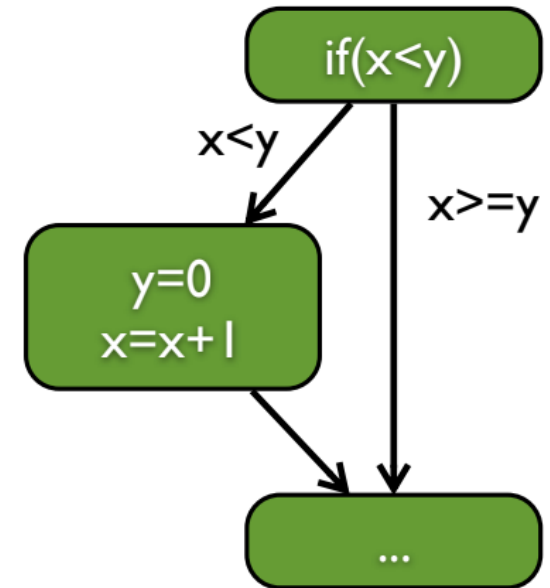


CFG: Câu lệnh if

```
if (x < y)
{
    y = 0;
    x = x + 1;
}
else
{
    x = y;
}
...
```



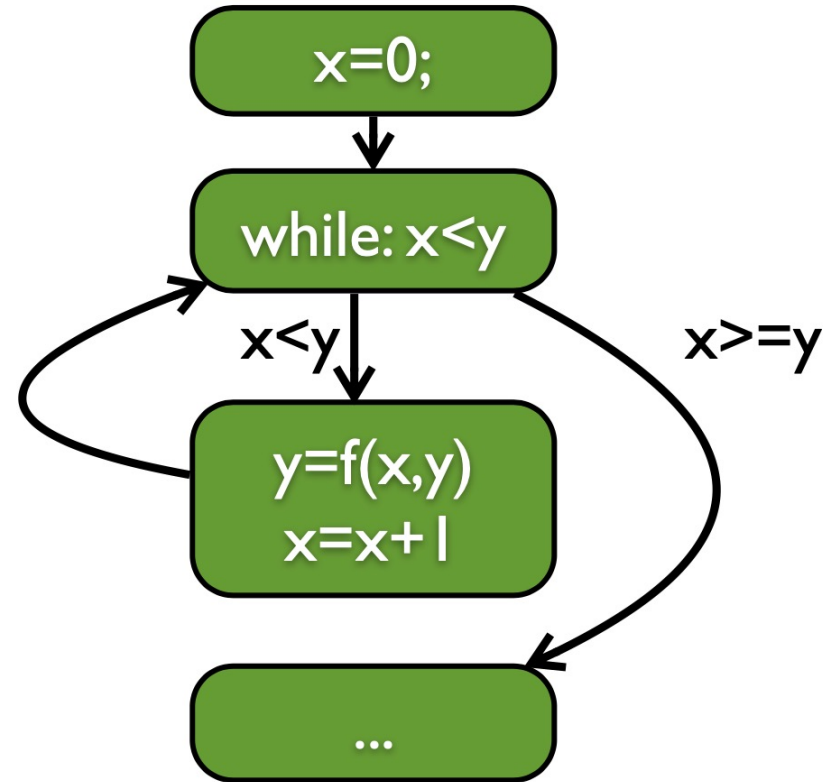
```
if (x < y)
{
    y = 0;
    x = x + 1;
}
...
```



CFG: Câu lệnh while/for

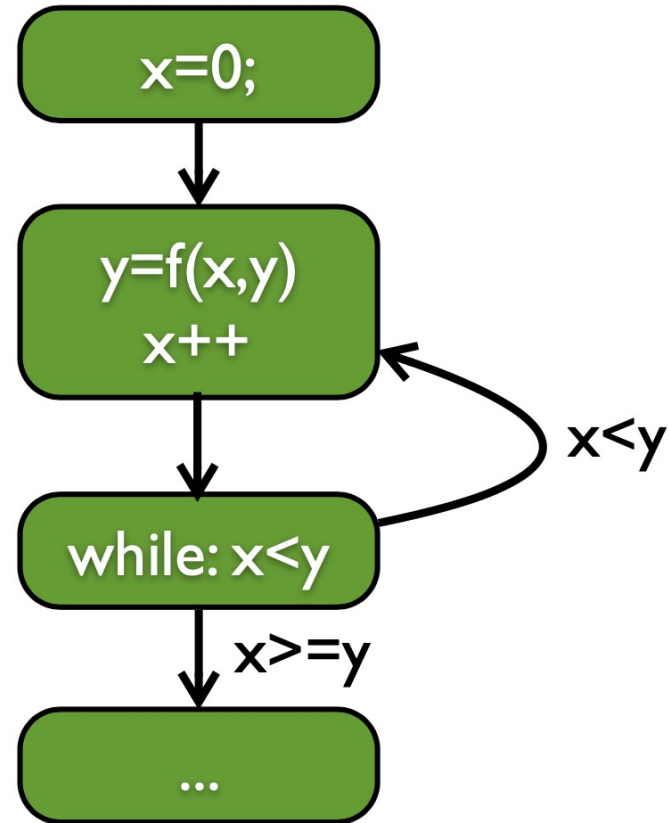
```
x=0;  
while (x < y)  
{  
    y = f (x, y);  
    x = x + 1;  
}  
...
```

```
for (x = 0; x < y; x++)  
{  
    y = f (x, y);  
}
```



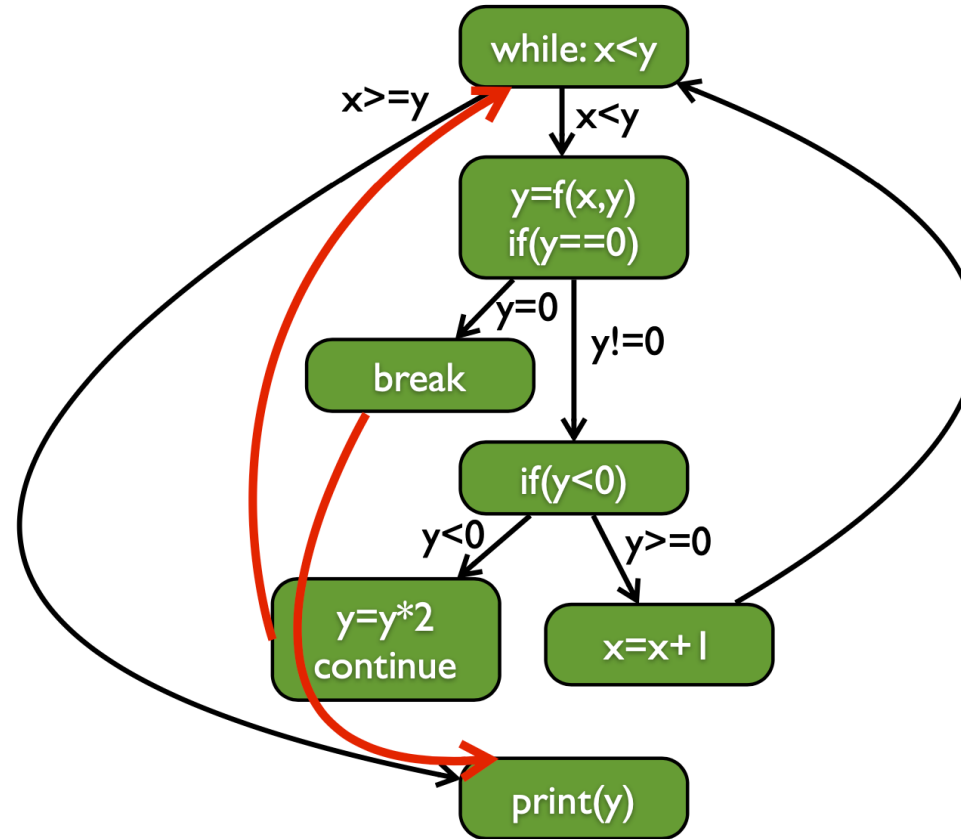
CFG: Câu lệnh while/for

```
x=0;  
do {  
    y = f (x, y);  
    x = x + 1;  
}  
while (x < y)  
...
```



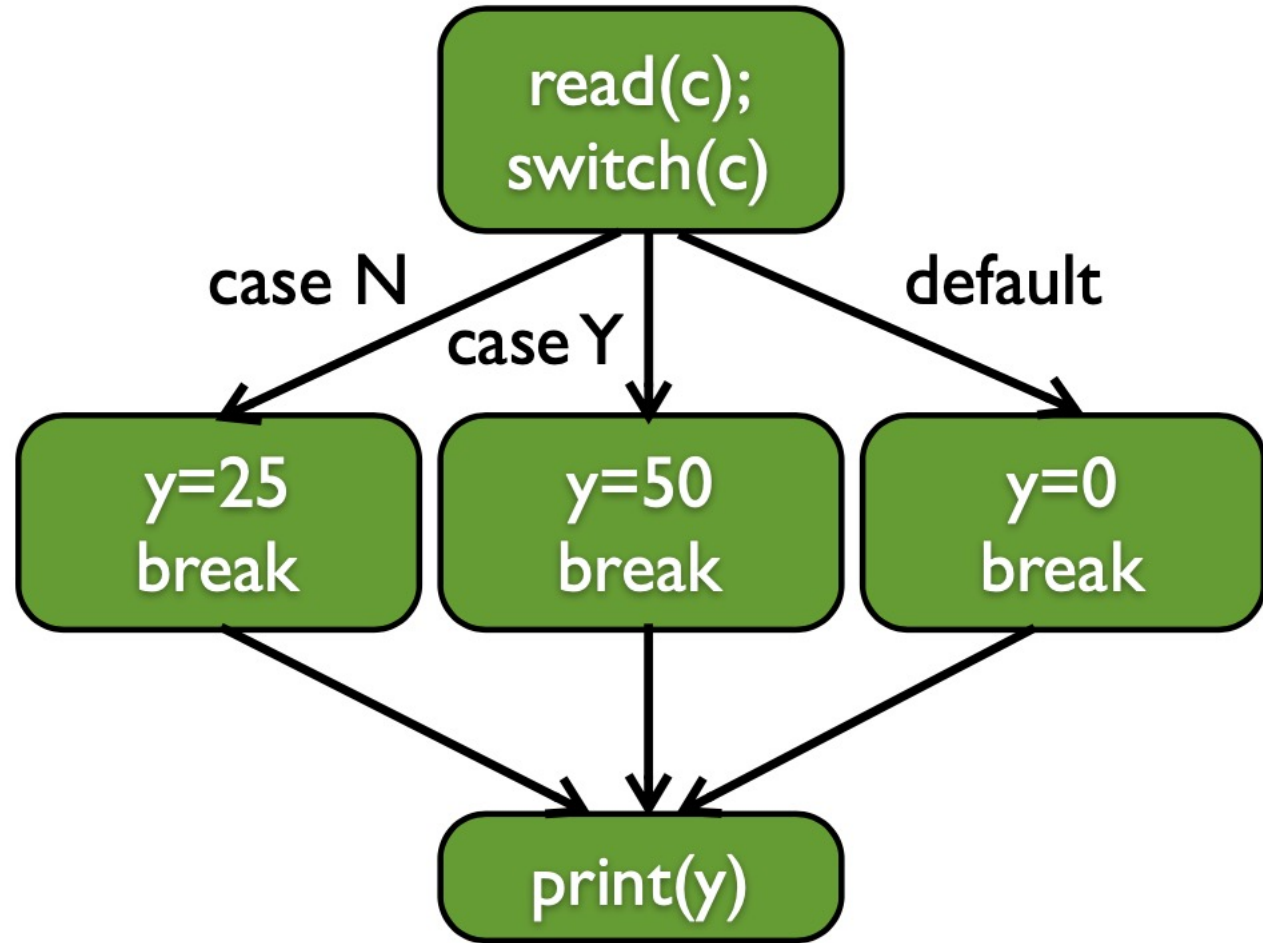
CFG: câu lệnh break và continue

```
while (x < y)
{
    y = f(x, y);
    if (y == 0) {
        break;
    } else if (y < 0) {
        y = y*2;
        continue;
    }
    x = x + 1;
}
print(y);
```



CFG: switch

```
read ( c );  
switch ( c )  
{  
    case 'N':  
        y = 25;  
        break;  
    case 'Y':  
        y = 50;  
        break;  
    default:  
        y = 0;  
        break;  
}  
print (y);
```



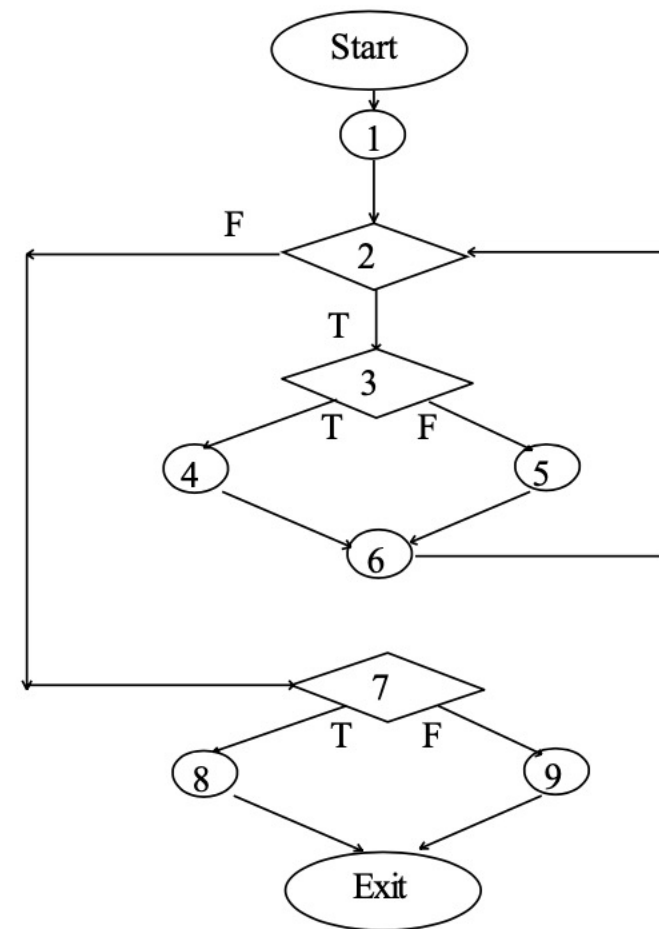
Đường đi trong đồ thị dòng điều khiển

- 1 đường đi (a path) được biểu diễn bởi một chuỗi các đỉnh từ đỉnh entry/start tới đỉnh exit/end

Ví dụ:

Start - 1 - 2(T) - 3(T)- 6 - 2(F)- 7(T)- 8 - Exit

Start - 1 - 2(F) - 7(F) - 9 - Exit



Ca kiểm thử – Test case

- Một **ca kiểm thử** kiểm tra một **đường đi hoàn chỉnh** (complete path) từ entry node tới exit node trên CFG
- Độ bao phủ (Coverage): Đo lường mức độ bao phủ (cover) của bộ kiểm thử với chương trình

Tiêu chí lựa chọn đường đi – Path selection criteria

- Với chương trình nhỏ: Có thể thực thi/test tất cả các đường đi có thể của chương trình
 - Với chương trình lớn: Thực thi/test tất cả các đường đi là không khả thi
- > Cần có tiêu chí hiệu quả để lựa chọn các đường đi phù hợp (giảm nỗ lực kiểm thử, tăng hiệu quả trong việc tìm lỗi)

Cần lựa chọn những đường đi nào để kiểm thử?

Tiêu chí lựa chọn đường đi – Path selection criteria

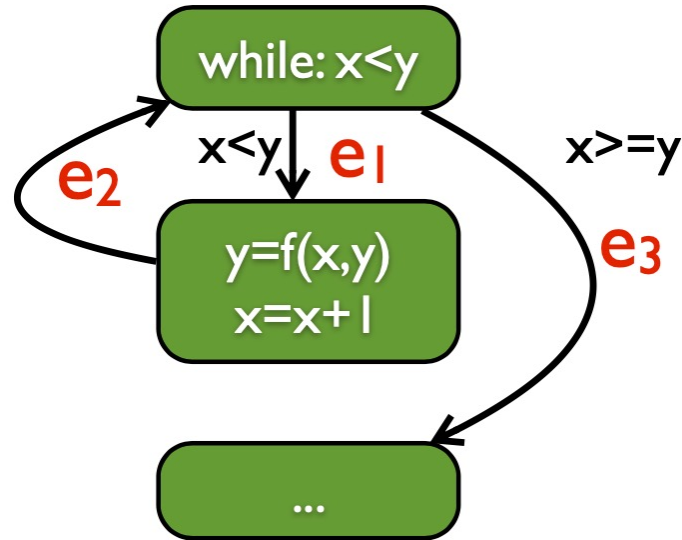
- Lựa chọn đường đi dựa trên độ phủ (coverage)
 - + Lựa chọn tất cả các đường đi
 - + Lựa chọn các đường đi để đạt độ phủ câu lệnh
 - + Lựa chọn các đường đi để đạt độ phủ nhánh
 - + Lựa chọn đường đi để đạt độ phủ điều kiện (predicate)

Phủ đường đi - path coverage

- **Tất cả** các paths trong một CFG được lựa chọn để kiểm thử.
- Tuy nhiên một chương trình có thể bao gồm vô số đường đi
- Lựa chọn tất cả các đường đi để kiểm thử là tiêu chí hiệu quả để xác định lỗi, tuy nhiên rất khó được tiêu chí này trong thực tế

Phủ đường đi - path coverage

```
while (x < y)
{
    y = f (x, y);
    x = x + 1;
}
...
```



Tất cả các đường đi có thể của chương trình:

e_3

$e_1e_2e_3$

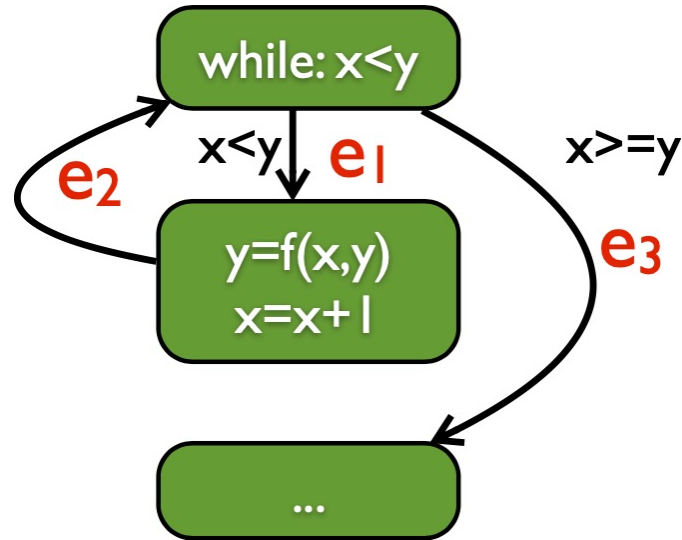
$e_1e_2e_1e_2e_3$

$e_1e_2e_1e_2e_1e_2e_3$

...

Phủ đường đi - path coverage

```
while (x < y)
{
    y = f(x, y);
    x = x + 1;
}
...
```



Tất cả các đường đi có thể của chương trình:

e_3

$e_1e_2e_3$

$e_1e_2e_1e_2e_3$

$e_1e_2e_1e_2e_1e_2e_3$

...

Để đạt được 100% độ phủ đường đi có thể không khả thi trong thực tế

Phủ cấp 1 (C1) – Statement coverage criterion

- Phủ một câu lệnh (statement coverage) s:
 - Lựa chọn đường đi sao cho bao gồm câu lệnh s
- **100% phủ câu lệnh (complete statement coverage):**
 - **Mỗi câu lệnh** trong chương trình được thực thi **ít nhất một lần** sau khi chạy các ca kiểm thử
- Vấn đề:
 - Lựa chọn ít paths mà có thể phủ được tất cả các đỉnh của CFG → để đạt độ phủ câu lệnh

Phủ cấp 1 (C1) – Statement coverage criterion

- Một số luật (rules) có thể áp dụng khi lựa chọn đường đi:
 - Lựa chọn đường đi ngắn (short paths)
 - Lựa chọn đường đi tăng dần (increasingly longer paths)
 - Lựa chọn các đường đi phức tạp

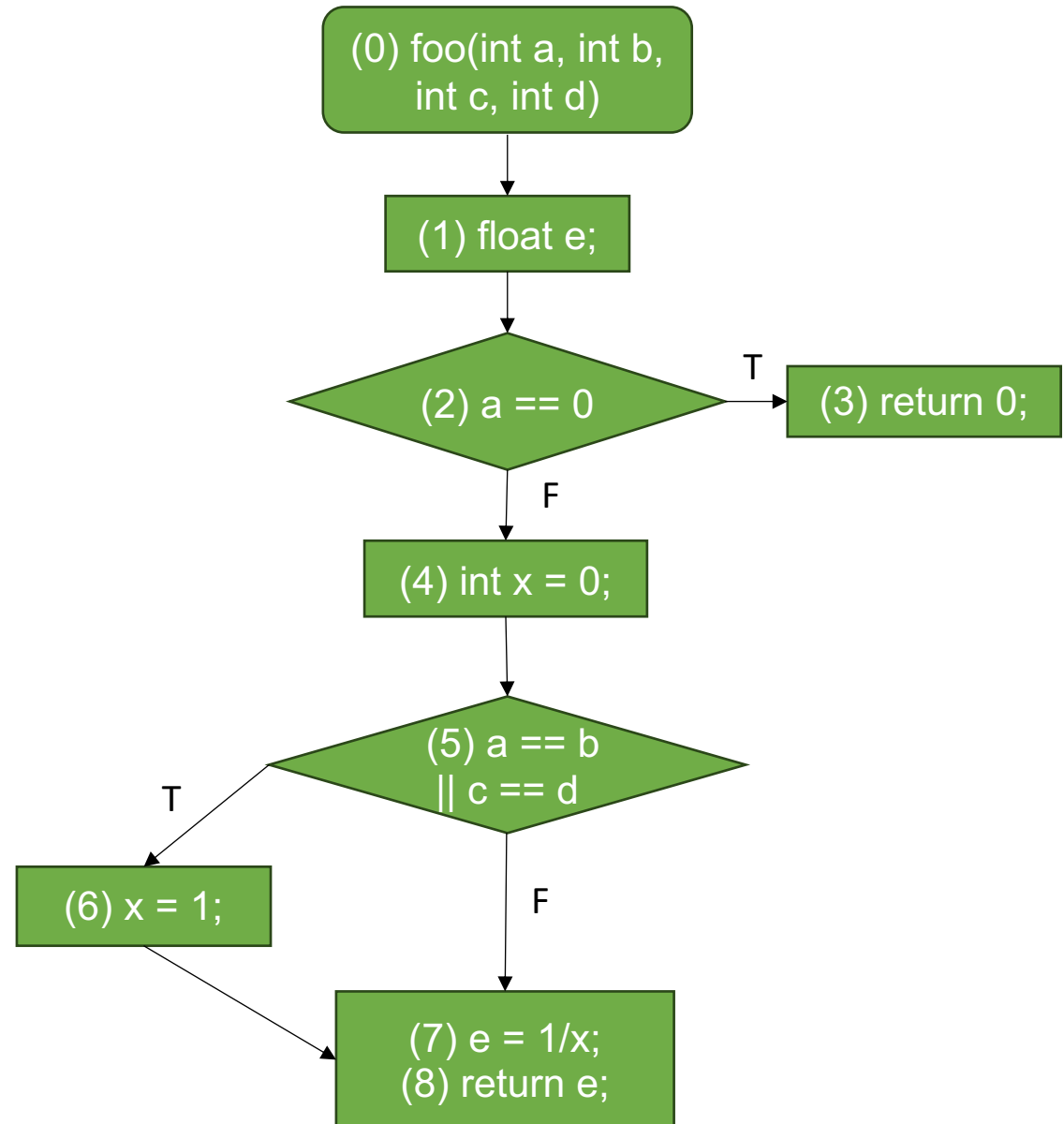
Ví dụ

```
0. float foo(int a, int b, int c, int d){  
1.   float e;  
2.   if (a == 0)  
3.     return 0;  
4.   int x = 0;  
5.   if ( (a==b) || ( c == d))  
6.     x = 1;  
7.   e = 1/x;  
8.   return e;  
9. }
```

Biểu diễn CFG cho hàm foo()

Ví dụ

```
0. float foo(int a, int b, int c, int d){  
1.   float e;  
2.   if (a == 0)  
3.     return 0;  
4.   int x = 0;  
5.   if ( (a==b) || ( c == d))  
6.     x = 1;  
7.   e = 1/x;  
8.   return e;  
9. }
```



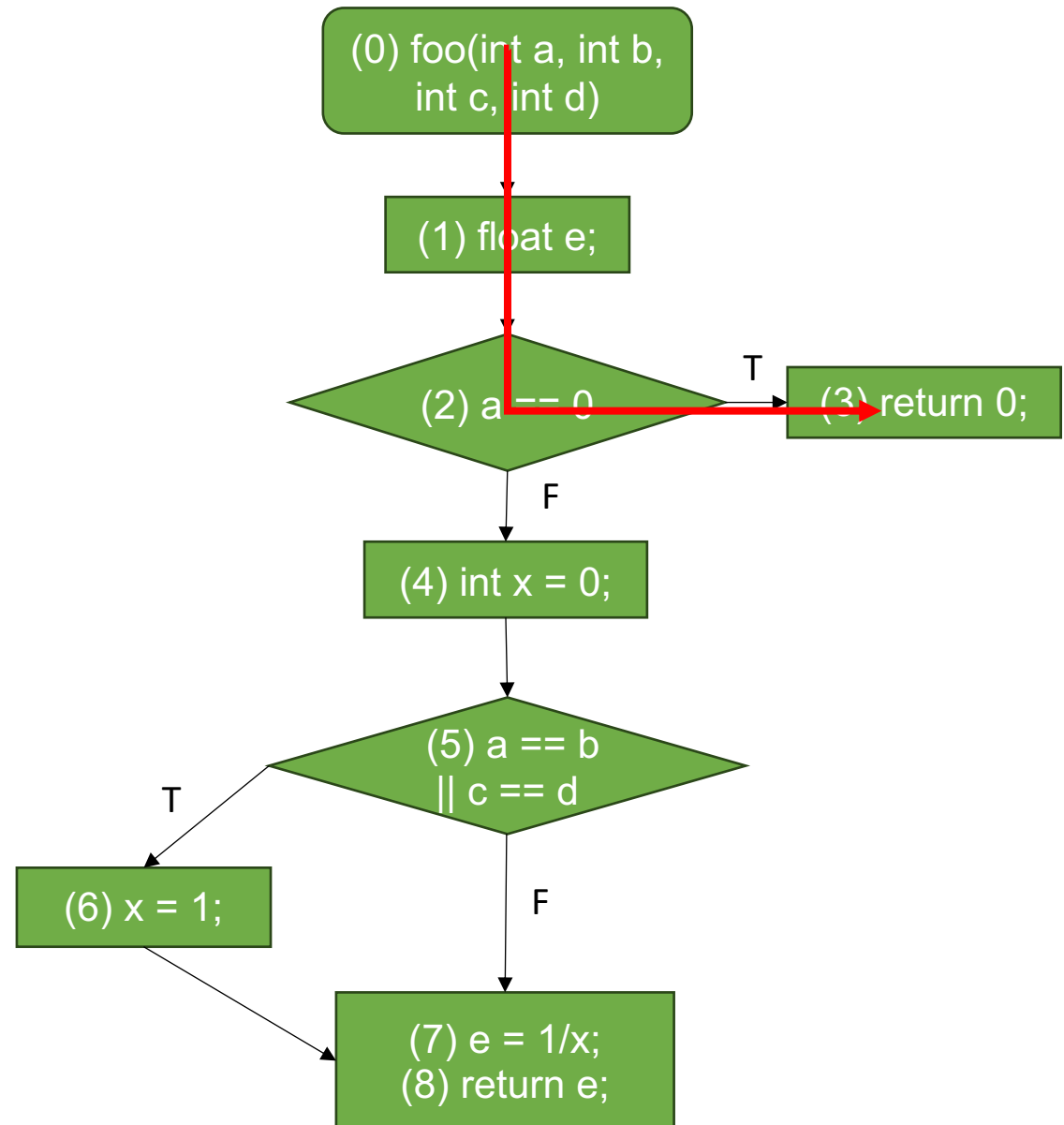
Ví dụ - Phủ câu lệnh

- Path 1:

0 – 1 – 2 (T) – 3

→ Test case: foo(0, 0, 0, 0)

Scov = ?



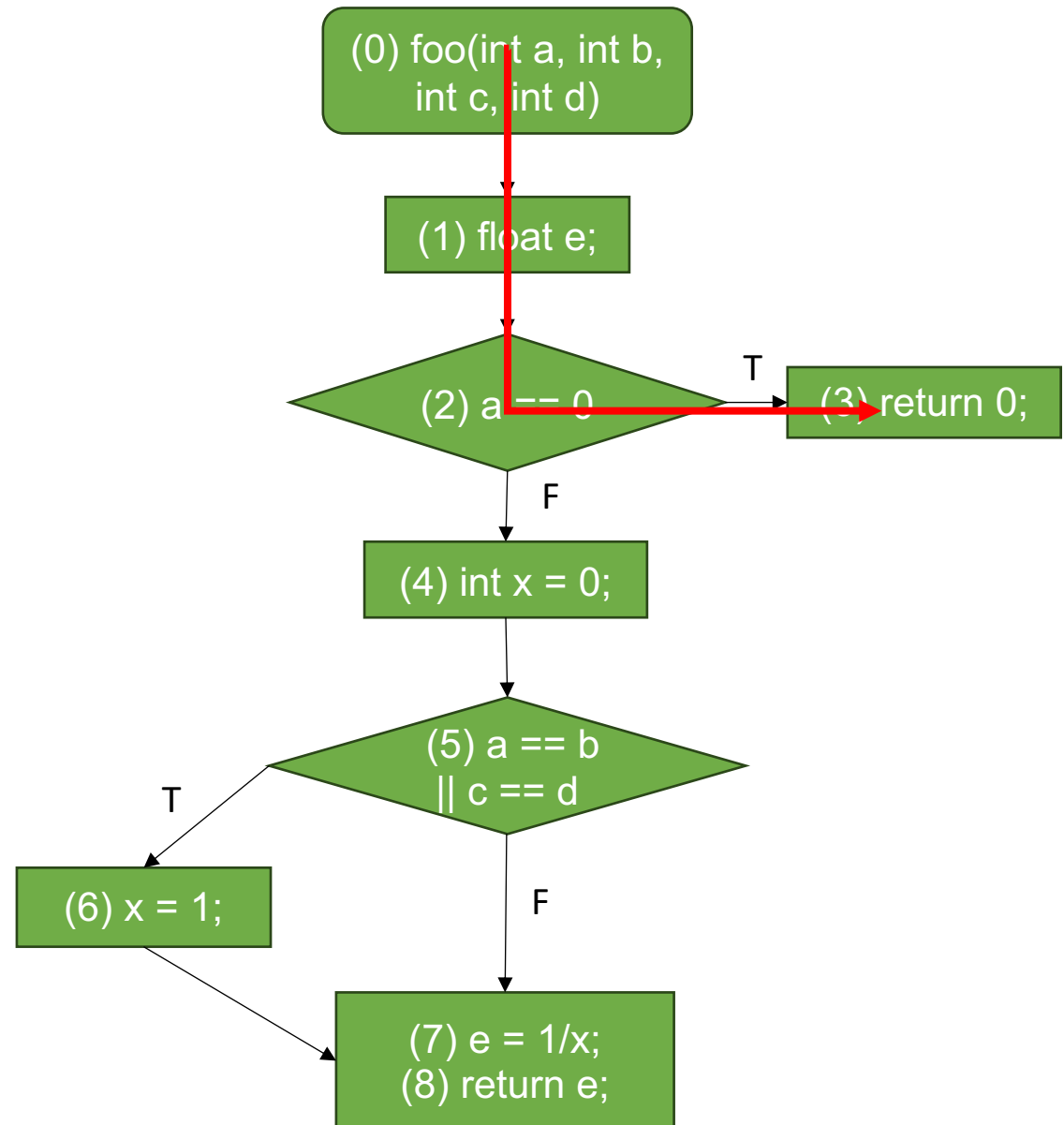
Ví dụ - Phủ câu lệnh

- Path 1:

0 – 1 – 2 (T) – 3

→ Test case: foo(0, 0, 0, 0)

$$\text{Scov} = 3/8 = 0.375$$



Ví dụ - Phủ câu lệnh

- Path 1:

0 – 1 – 2 (T) – 3

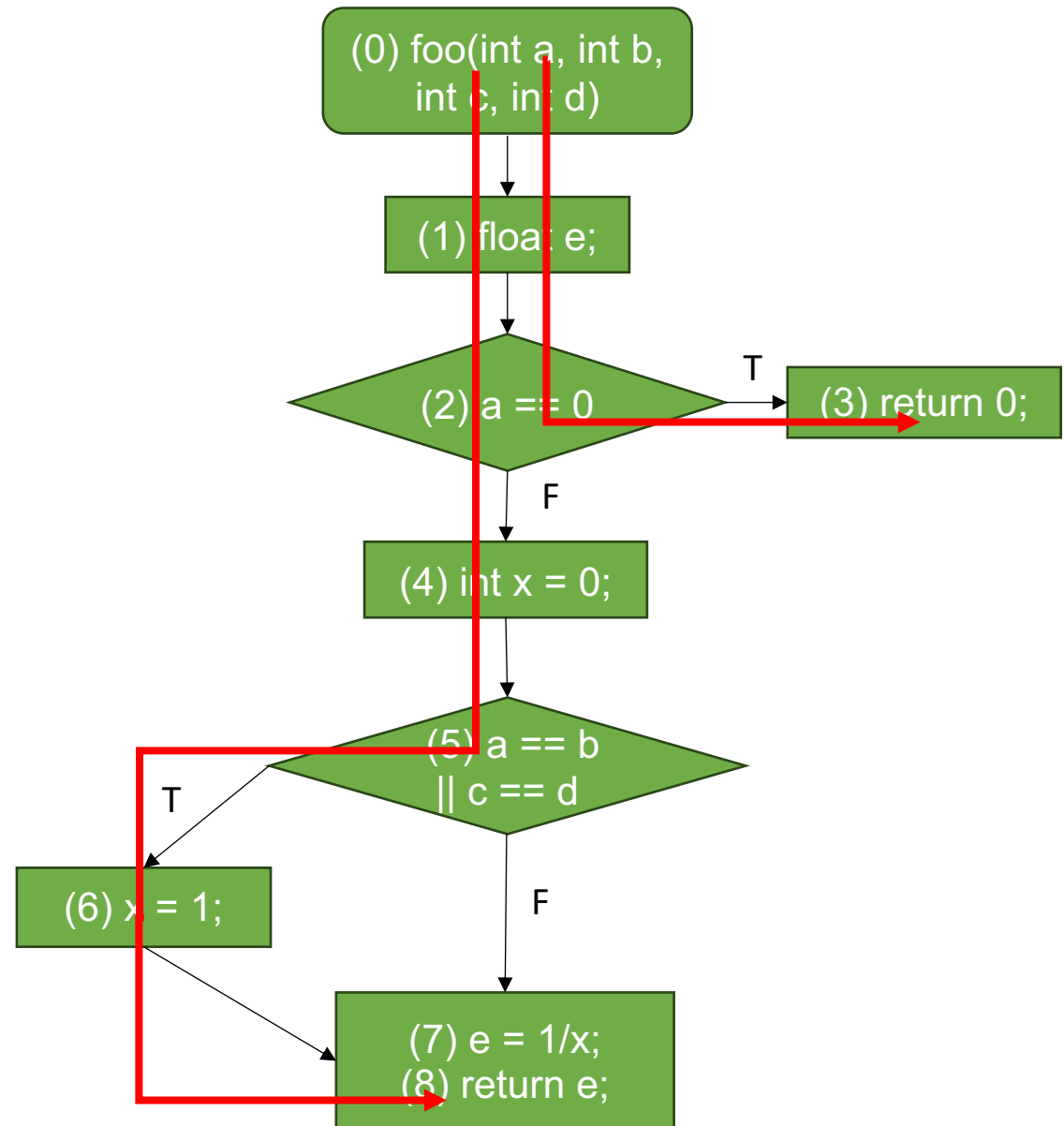
→ Test case: foo(0, 0, 0, 0)

- Path 2

0 – 1 – 2 (F) – 4 – 5(T) – 6 – 7 – 8

→ Test case: foo(1, 1, 0, 1)

$$\text{Scov} = 8/8 = 1$$



Ví dụ - Phủ câu lệnh

- Path 1:

0 – 1 – 2 (T) – 3

→ Test case: foo(0, 0, 0, 0)

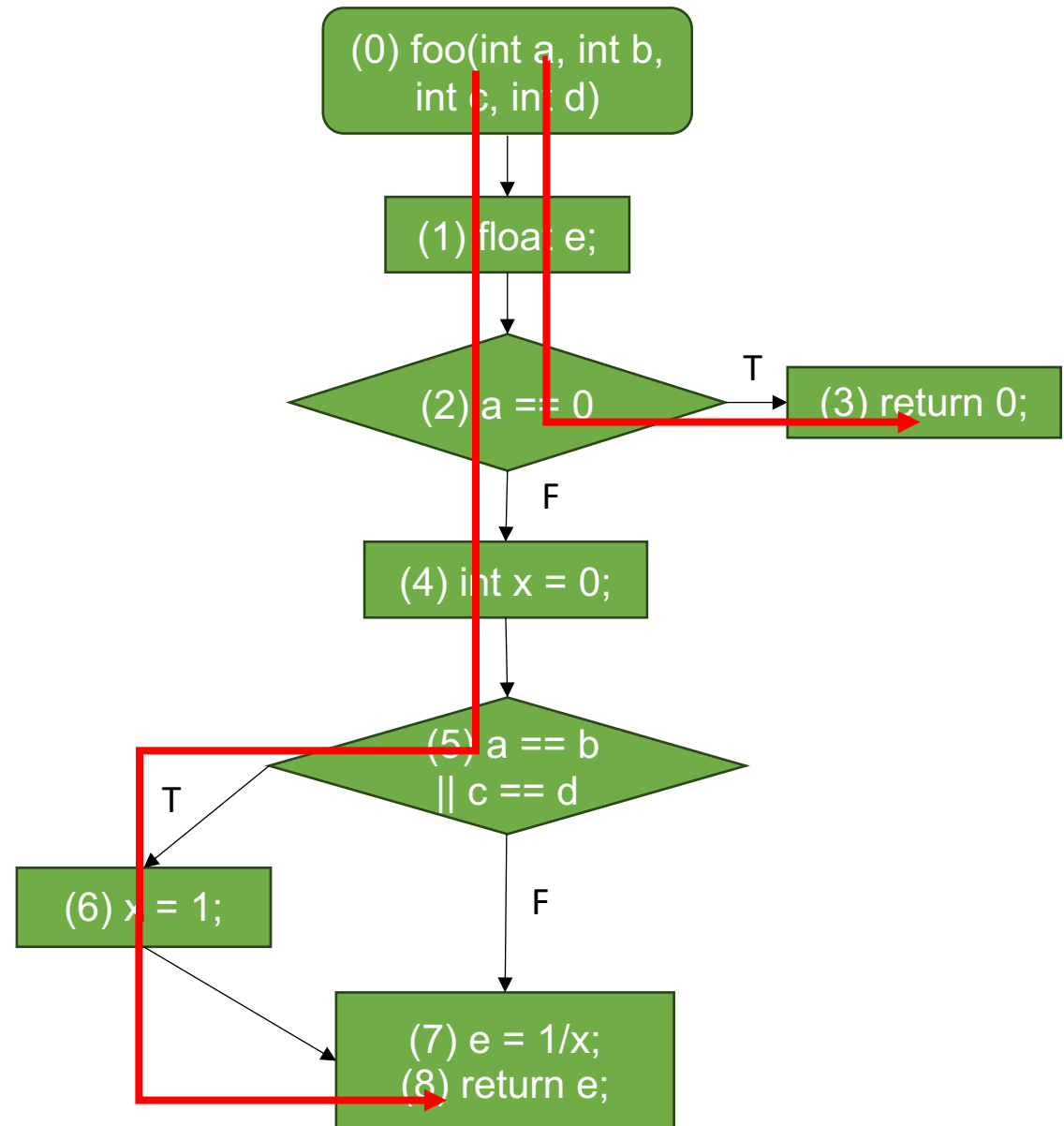
- Path 2

0 – 1 – 2 (F) – 4 – 5(T) – 6 – 7 – 8

→ Test case: foo(1, 1, 0, 1)

$$\text{Scov} = 8/8 = 1$$

Chương trình này có lỗi gì và bộ 2 test cases này có phát hiện được không?



Phủ cấp C1 - Statement coverage

- Bao phủ dòng lệnh **không nói được vòng lặp có kết thúc không** – chỉ biết được là thân vòng lặp đã được thực hiện
- Bao phủ dòng lệnh **không quan tâm đến các quyết định logic**
- Thường chỉ rất ít ca kiểm thử đã có thể đạt được bao phủ dòng lệnh
→ khó chấp nhận để phát hành, đưa vào sử dụng

Phủ cấp 2 (C2) – Branch Coverage Criterion

- **Nhánh (Branch):** cạnh ra (out-going edges) của node

Ví dụ: Khối xử lý - Node chỉ có 1 nhánh
Khối quyết định – Node có 2 nhánh
Node exit – không có nhánh

Phủ cấp 2 (C2) – Branch Coverage Criterion

- Phủ 1 nhánh (covering a branch):
 - Lựa chọn 1 đường đi mà bao gồm nhánh đó
- **Đạt 100% độ phủ nhánh** (Complete branch coverage):
 - Lựa chọn các đường đi sao cho **mỗi nhánh đều được đi qua ít nhất một lần**

Ví dụ - Phủ nhánh

- Path 1:

0 – 1 – 2 (T) – 3

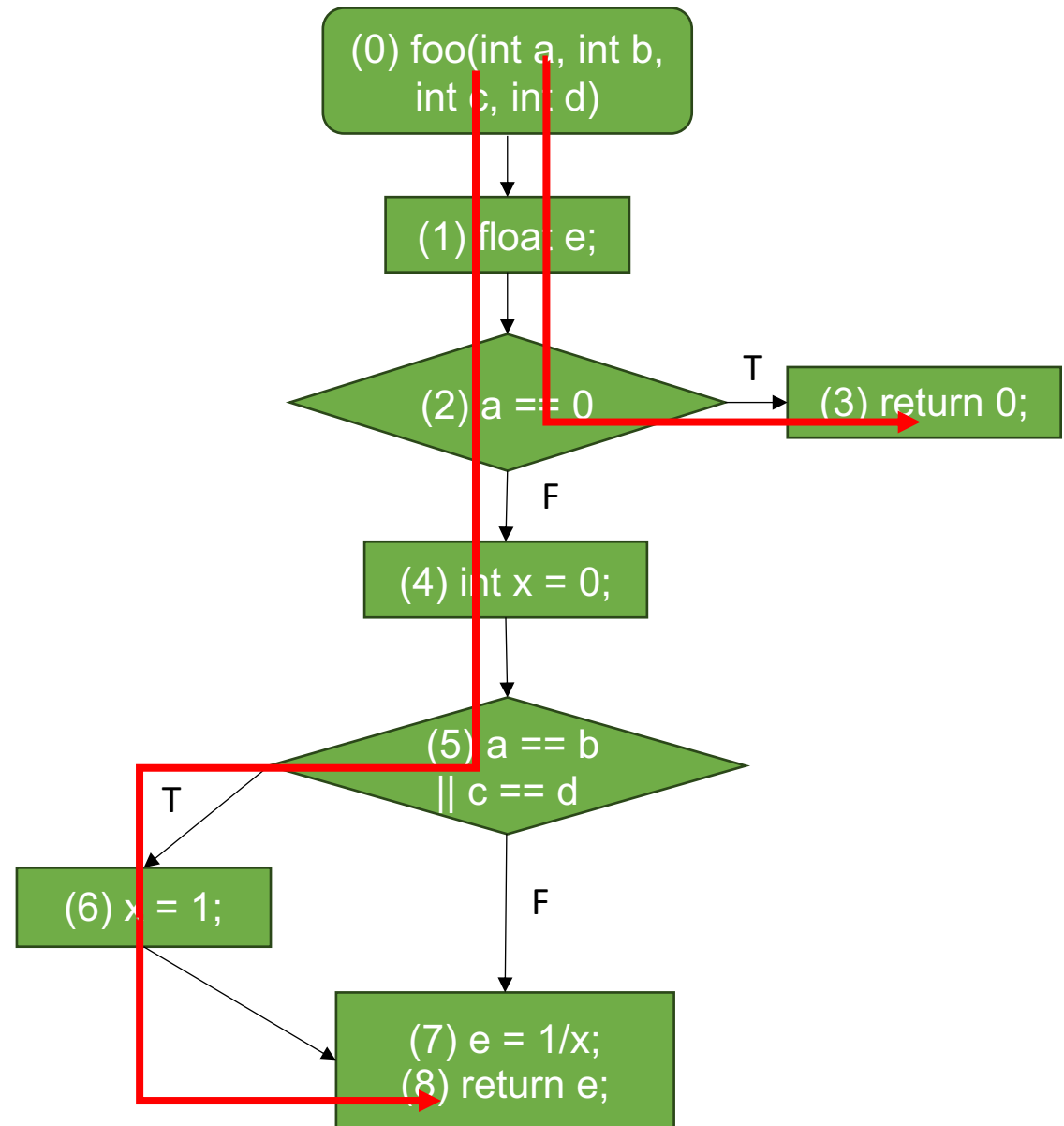
→ Test case: foo(0, 0, 0, 0)

- Path 2

0 – 1 – 2 (F) – 4 – 5(T) – 6 – 7 – 8

→ Test case: foo(1, 1, 0, 1)

Bcov = ?



Ví dụ - Phủ nhánh

- Path 1:

0 – 1 – 2 (T) – 3

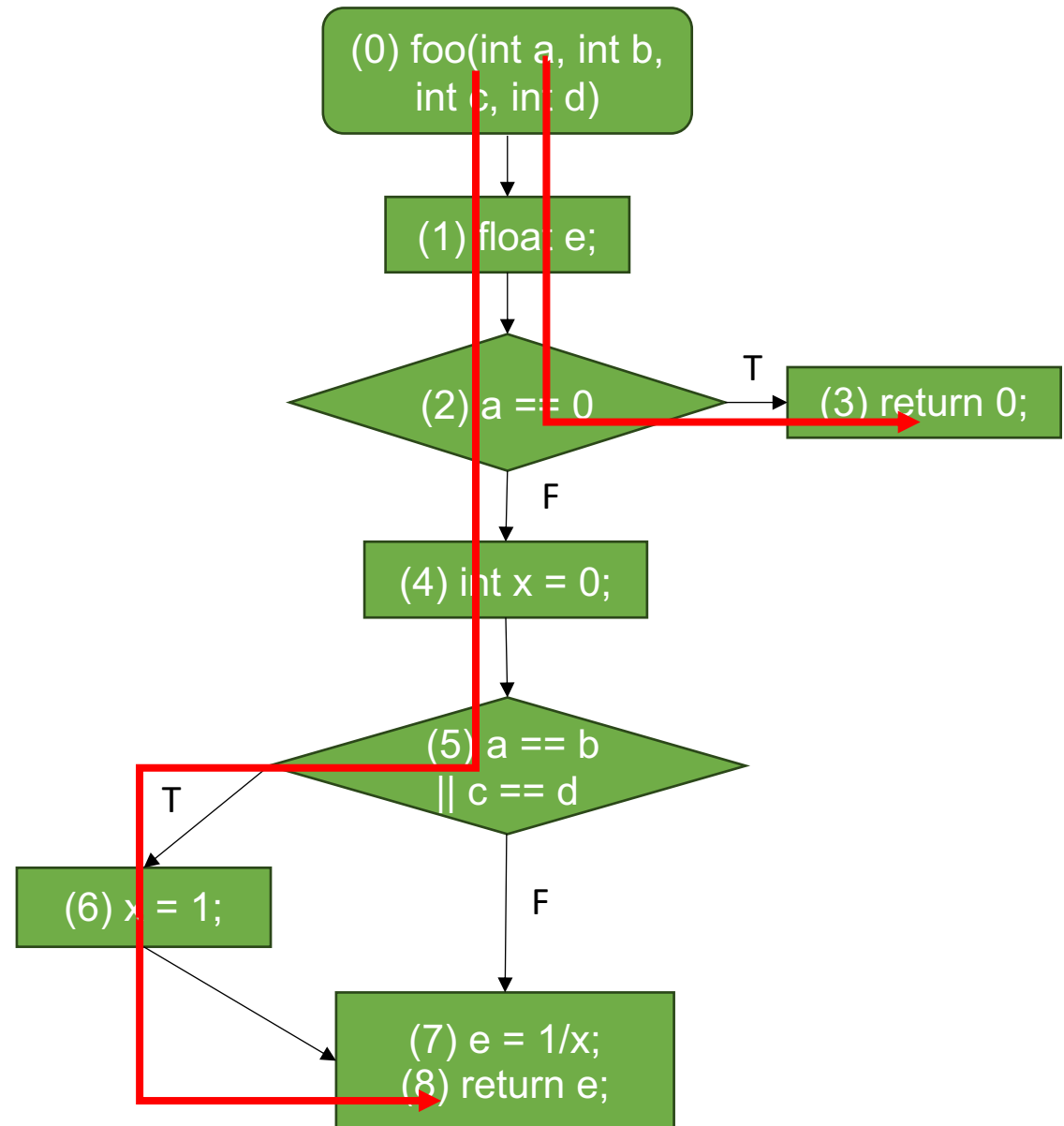
→ Test case: foo(0, 0, 0, 0)

- Path 2

0 – 1 – 2 (F) – 4 – 5(T) – 6 – 7 – 8

→ Test case: foo(1, 1, 0, 1)

$$Bcov = \frac{3}{4} = 0.75$$



Ví dụ - Phủ nhánh

- Path 1:

0 – 1 – 2 (T) – 3

→ Test case: foo(0, 0, 0, 0)

- Path 2

0 – 1 – 2 (F) – 4 – 5(T) – 6 – 7 – 8

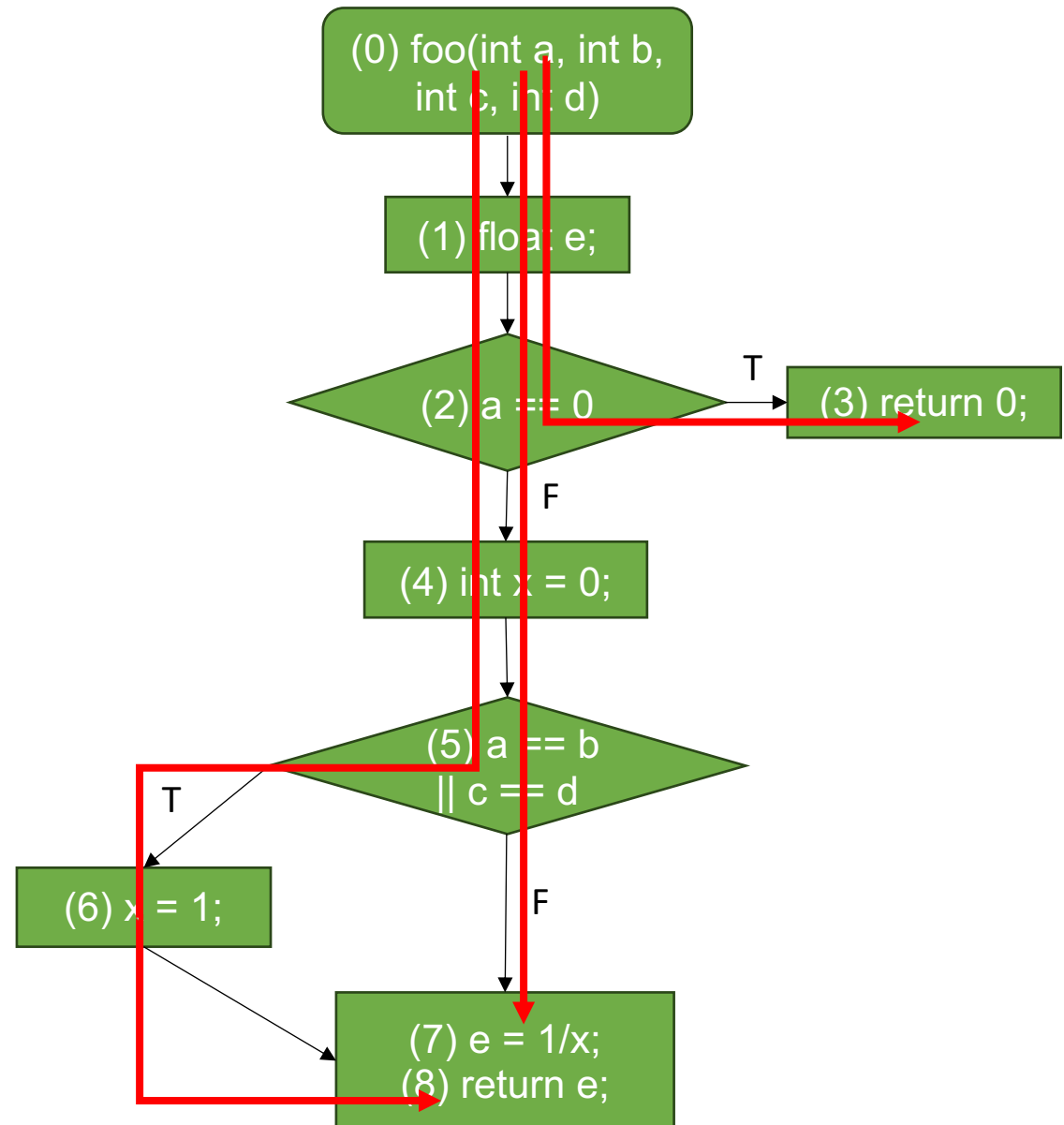
→ Test case: foo(1, 1, 0, 1)

- Path 3

0 – 1 – 2 (F) – 4 – 5(F) – 7 – 8

→ Test case 3: foo(1, 0, 1, 0)

$$\text{Bcov} = 4/4 = 1$$



Phủ cấp 2 (C2) – Branch Coverage Criterion

- **Ưu điểm:**
 - Đơn giản, nhưng không có các vấn đề của bao phủ câu lệnh
- **Nhược điểm:**
 - Vẫn gây thiếu ca kiểm thử với một số ngôn ngữ cho phép xử lý tắt (short-circuit) các phép toán logic
 - ví dụ: if (true || X) thì X bị bỏ qua

Phủ cấp 3 – Predicate Coverage Criterion

- Với các điều kiện phức tạp, việc chỉ quan tâm đến giá trị đúng sai là không đủ để kiểm tra tính đúng đắn của chương trình
- **Phủ cấp 3:** Kiểm thử sao cho **mỗi điều kiện con** của từng điểm quyết định đều được **thực hiện ít nhất một lần cho trường hợp True và False**

Ví dụ - Phủ điều kiện

- Path 1:

0 – 1 – 2 (T) – 3

→ Test case 1: foo(0, 0, 0, 0)

- Path 2

0 – 1 – 2 (F) – 4 – 5(T) – 6 – 7 – 8

0 – 1 – 2 (F) – 4 – 5a(T) – 6 – 7 – 8

→ Test case 2: foo(1, 1, 0, 1)

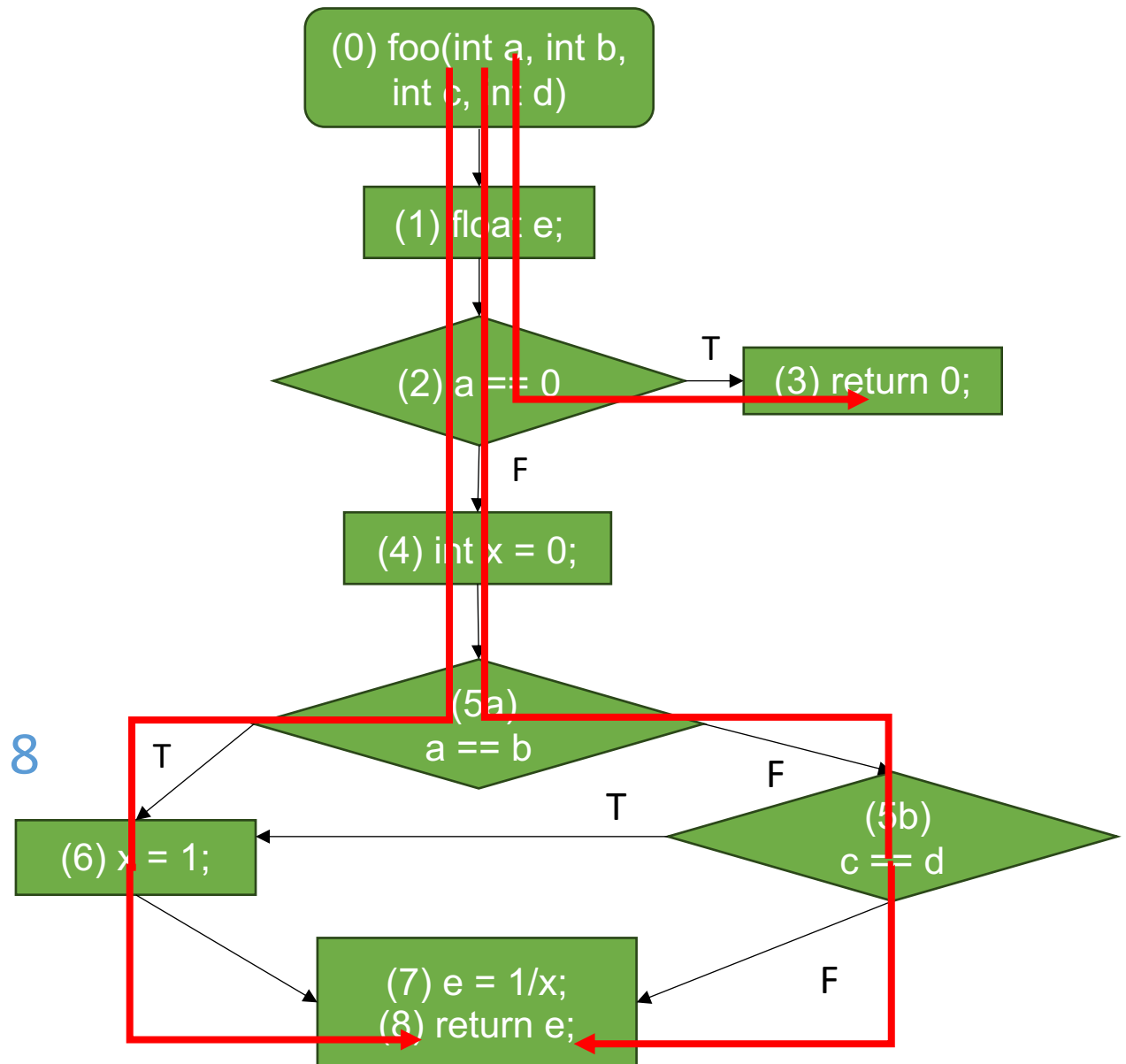
- Path 3

0 – 1 – 2 (F) – 4 – 5(F) – 7 – 8

0 – 1 – 2 (F) – 4 – 5a(F) – 5b (F) – 7 – 8

→ Test case 3: foo(1, 0, 1, 0)

Pcov = ?



Ví dụ - Phủ điều kiện

- Path 1:

0 – 1 – 2 (T) – 3

→ Test case 1: foo(0, 0, 0, 0)

- Path 2

0 – 1 – 2 (F) – 4 – 5(T) – 6 – 7 – 8

0 – 1 – 2 (F) – 4 – 5a(T) – 6 – 7 – 8

→ Test case 2: foo(1, 1, 0, 1)

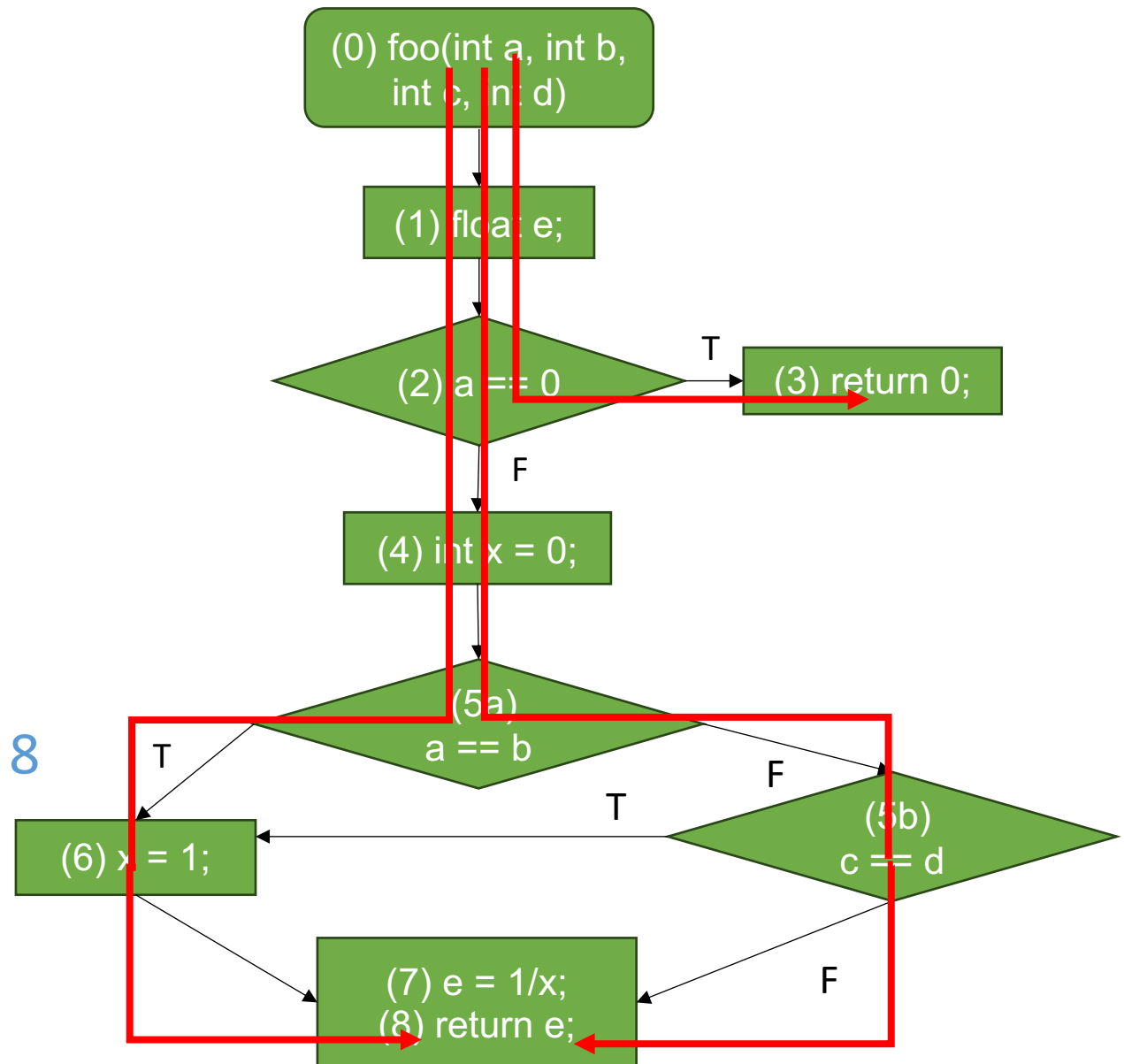
- Path 3

0 – 1 – 2 (F) – 4 – 5(F) – 7 – 8

0 – 1 – 2 (F) – 4 – 5a(F) – 5b (F) – 7 – 8

→ Test case 3: foo(1, 0, 1, 0)

$P_{cov} = 5/6 = 0.83$



Ví dụ - Phủ điều kiện

- Path 1:

0 – 1 – 2 (T) – 3

→ Test case 1: foo(0, 0, 0, 0)

- Path 2

0 – 1 – 2 (F) – 4 – 5(T) – 6 – 7 – 8

0 – 1 – 2 (F) – 4 – 5a(T) – 6 – 7 – 8

→ Test case 2: foo(1, 1, 0, 1)

- Path 3

0 – 1 – 2 (F) – 4 – 5(F) – 7 – 8

0 – 1 – 2 (F) – 4 – 5a(F) – 5b (F) – 7 – 8

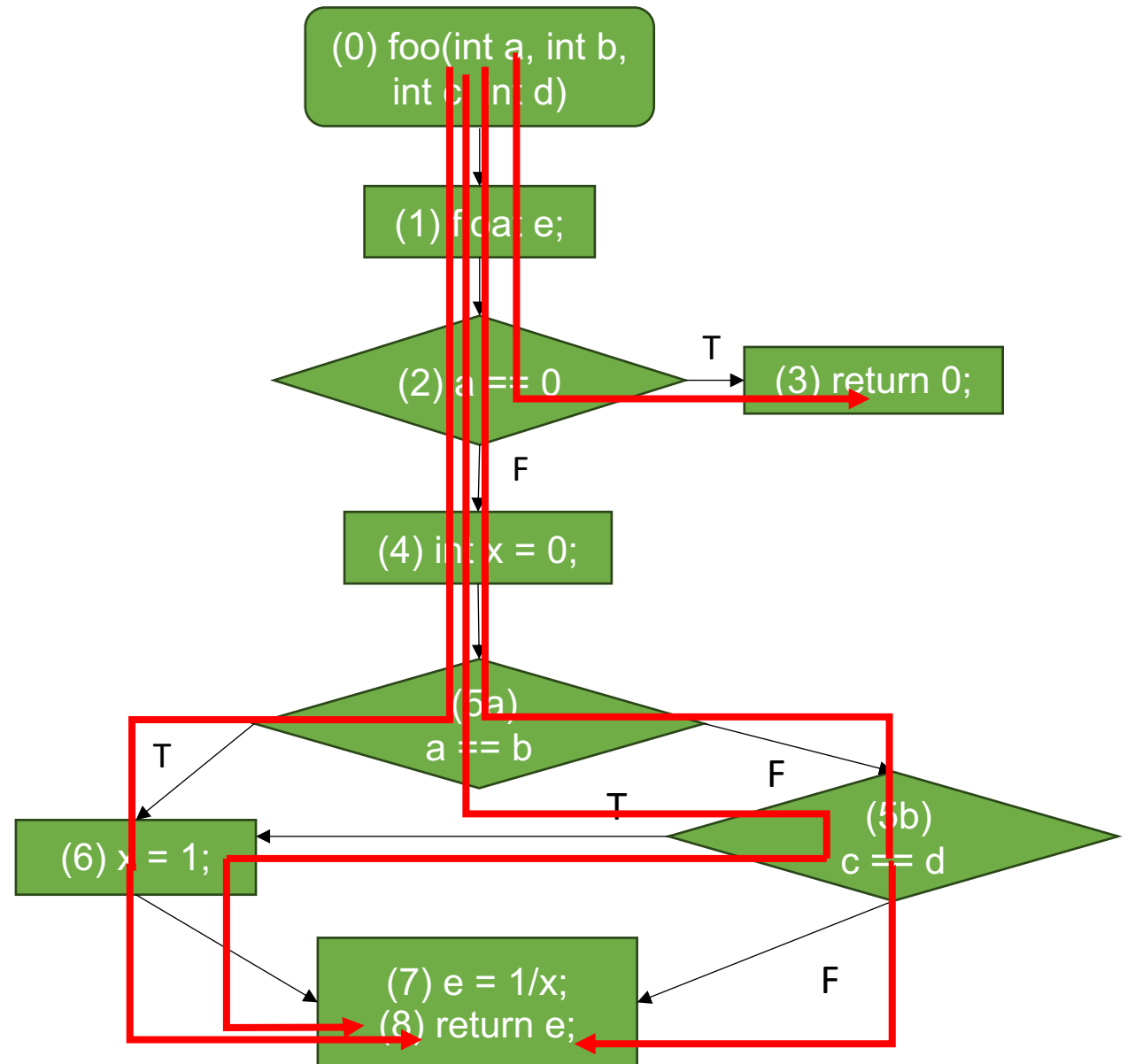
→ Test case 3: foo(1, 0, 1, 0)

- Path 4

0 – 1 – 2 (F) – 4 – 5a(F) – 5b (T) – 6 – 7 – 8

→ Test case 4: foo(1, 0, 1, 1)

$P_{cov} = 6/6 = 1$



Bao phủ vòng lặp

Với mỗi đơn vị chương trình có vòng lặp, chúng ta cần quan tâm đến 3 trường hợp sau:

- **Lệnh lặp đơn giản:**

- Đơn vị chương trình chỉ chứa đúng 1 vòng lặp (thân của vòng lặp không chứa vòng lặp khác)

- **Lệnh lặp liền kề:**

- Đơn vị chương trình chỉ chứa các lệnh lặp kế tiếp nhau

- **Lệnh lặp lồng nhau:**

- Đơn vị chương trình chỉ chứa các vòng lặp chứa các lệnh lặp khác

Bao phủ vòng lặp

- Với mỗi vòng lặp đơn giản, phép đo này đảm bảo:
 - Vòng lặp được thực hiện 0 lần
 - Vòng lặp được thực hiện 1 lần
 - Vòng lặp được thực hiện 2 lần
 - Vòng lặp được thực hiện k lần, $2 < k < n - 1$, với n là số lần lặp tối đa
 - Vòng lặp được thực hiện $n - 1$ lần
 - Vòng lặp được thực hiện n lần
 - Vòng lặp được thực hiện $n + 1$ lần (Nhiều trường hợp không sinh được ca kiểm thử này)
- Với trường hợp không xác định được số lần lặp tối đa: Các ca kiểm thử cần đảm bảo 4 tiêu chí đầu tiên
- Với chương trình có vòng lặp liên kề: tiến hành tuần tự từ trên xuống
- Với chương trình có vòng lặp lồng nhau: tiến hành tuần tự từ trong ra ngoài

Bao phủ hàm

- Phép đo này cho ta biết mọi hàm trong chương trình đều được gọi
- Điều này hữu ích khi kiểm thử sơ bộ, để đảm bảo tính đúng đắn các “vùng” của phần mềm

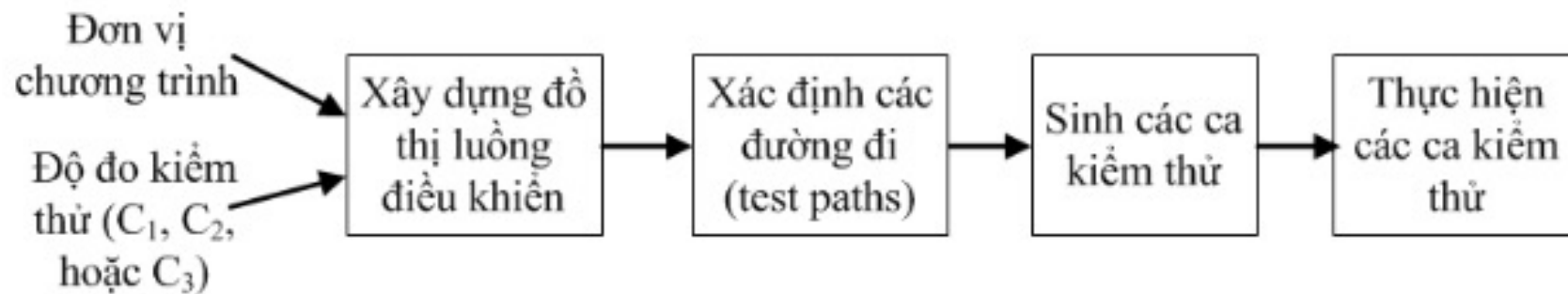
Bao phủ gọi

- Phép đo này cho biết mọi lời gọi hàm trong chương trình đều được thực hiện
 - Thích hợp khi muốn kiểm tra giao diện giữa các mô-đun
- Còn được gọi là bao phủ cặp gọi (call pair)

Kiểm thử dựa trên độ đo

- Với mỗi chương trình, CFG ứng với độ đo C1 và C2 là giống nhau, trong khi chúng khác với CFG ứng với độ đo C3
- Với mỗi đơn vị chương trình và mỗi độ đo kiểm thử, chúng ta tiến hành xây dựng đồ thị dòng điều khiển tương ứng
- Xác định các đường đi của chương trình sao cho khi chúng thực hiện thì độ đo kiểm thử tương ứng được thoả mãn
- Với mỗi đường đi sinh ra một ca kiểm thử tương ứng

Kiểm thử dựa trên độ đo



Hình 6.4: Quy trình kiểm thử đơn vị chương trình dựa trên độ đo.

Tổng kết

- Dòng điều khiển
- Kiểm thử dòng điều khiển là một trong những phương pháp kiểm thử quan trọng nhất trong kiểm thử hộp trắng
 - Kiểm thử đường đi của nó tương ứng với các dòng điều khiển
- Các độ đo kiểm thử -> Sinh ra các đường đi độc lập -> Mỗi đường đi sẽ sinh ra một ca kiểm thử
- Sinh đầu vào cho các ca kiểm thử: Chọn một bộ đầu vào sao cho thoả mãn các điểm quyết định có trong đường đi tương ứng -> tính toán giá trị đầu ra

Bài tập

1. Trình bày các bước nhằm kiểm thử một đơn vị chương trình theo phương pháp kiểm thử dòng điều khiển với một độ đo kiểm thử cho trước

9. Cho hàm được viết bằng ngôn ngữ C như Đoạn mã 6.1.

Đoạn mã 6.1: Mã nguồn của hàm Grade

```
char Grade(int score){

    int res;
    if(score < 0 || score > 10)
        return 'I';
    if(score >= 9)
        res = 'A';
    else
        if(score >= 8)
            res = 'B';
        else
            if(score >= 6.5)
                res = 'C';
            else
                if(score >= 5)
                    res = 'D';
                else
                    res = 'F';

    return res;
}
```

- Hãy xây dựng đồ thị dòng điều khiển cho hàm Grade ứng với độ đo C_1 và C_2 .
- Hãy sinh các đường đi và các ca kiểm thử với độ đo C_1 .
- Hãy sinh các đường đi và các ca kiểm thử với độ đo C_2 .

13. Cho hàm được viết bằng ngôn ngữ C như Đoạn mã 6.5.

Đoạn mã 6.5: Mã nguồn của hàm Sum

```
int Sum(int a[], int n){  
    int i, total = 0;  
    for(i=0; i<n; i++)  
        total = total + a[i];  
    return total;  
}
```

- Hãy xây dựng đồ thị dòng điều khiển cho hàm Sum ứng với độ đo C_1 và C_2 .
- Hãy sinh các đường đi và các ca kiểm thử với độ đo C_1 .
- Hãy sinh các đường đi và các ca kiểm thử với độ đo C_2 .
- Hãy sinh các ca kiểm thử để kiểm thử vòng lặp `for`.

```
string foo(int x){  
    string res;  
    switch(x):  
        case 65:  
            res = "A";  
            break;  
        case 66:  
            res = "B";  
            break;  
        case 67:  
            res = "C";  
            break;  
        default:  
            res = "haven't check";  
    return res;  
}
```

- Xây dựng đồ thị dòng điều khiển cho hàm foo ứng với độ đo C2
- Hãy sinh các đường đi và các ca kiểm thử ứng với độ đo C2

5. Kiểm thử chương trình của bạn với độ đo kiểm thử C2

Các sản phẩm cần nộp

- Bài làm cho toàn bộ các bài tập trong slide
- Báo cáo phân tích, thiết kế các ca kiểm thử, và kiểm thử chương trình của bạn với độ phủ C2
- Mã nguồn và test cases